GUIA VISUAL 077

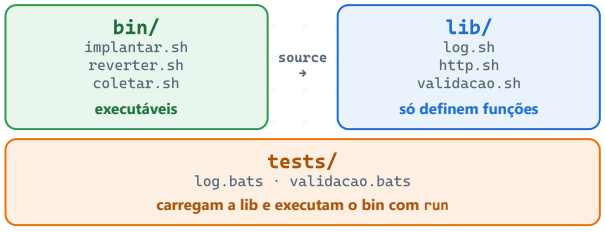
# ESTRUTURANDO SCRIPTS GRANDES E BIBLIOTECAS

*O `deploy.sh` tinha 1.400 linhas. Ninguém o escreveu: ele **cresceu** — trinta linhas em 2021 e um trecho novo a cada incidente, sempre colado no fim, sempre por alguém com pressa. Quatro funções de log com nomes diferentes, dezenove `curl` ligeiramente distintos, e no meio uma linha que ninguém ousava tocar: `# NAO REMOVER -- quebra a producao (nao sabemos por que)`. **Cada acréscimo isolado era razoável; ninguém parou para reorganizar.***

**077**
GUIAS LINUX

## AS OITO CAMADAS DE UM SCRIPT SÉRIO · E A DIREÇÃO DA DEPENDÊNCIA

| | | |
|---|---|---|
| 1 | **shebang + cabeçalho de uso** | a ajuda sai daqui por `sed` — nunca dessincroniza |
| 2 | **`set -Eeuo pipefail`**<br>**`shopt -s inherit_errexit`** | antes de qualquer comando que possa falhar — nem a leitura da biblioteca escapa |
| 3 | **constantes `readonly`** | caminhos, códigos de saída, versão. Atribuição acidental vira erro |
| 4 | **estado global mutável** | o mínimo possível: tudo o mais é `local` dentro de função |
| 5 | **`source` das bibliotecas** | **depois** das constantes — a biblioteca pode usá-las |
| 6 | **funções: genéricas → específicas** | uma responsabilidade cada, nome `verbo_substantivo` |
| 7 | **`principal()`** | só orquestra; não contém lógica de negócio |
| 8 | **guarda do ponto de entrada** | `[[ "${BASH_SOURCE[0]}" == "$0" ]]` — é o que torna o arquivo testável |




**A dependência aponta numa direção só:** `bin/` usa `lib/`; `lib/` não conhece `bin/`. Uma biblioteca que chama função definida no executável cria um **ciclo** que impede testar qualquer um dos dois isoladamente — e é o começo do arquivo de 1.400 linhas, porque a única forma de fazer funcionar passa a ser juntar tudo.

A ordem das camadas não é estética: **cada uma depende só das anteriores.** E o `RAIZ` merece atenção própria — `RAIZ="$(cd -- "$(dirname -- "$(readlink -f -- "${BASH_SOURCE[0]}")")/.." && pwd)"`. O `BASH_SOURCE` é o caminho **do arquivo**, não o do processo, então funciona igual quando o script é chamado por caminho relativo, por link simbólico no `PATH` ou por outro script; o `readlink -f` resolve o link. Usar `$0` ou `$PWD` aqui é o defeito que só aparece quando alguém chama de outro diretório — e aí a mensagem é obscura. **As cinco propriedades de uma biblioteca bem comportada:** (1) **não executa nada** ao ser carregada — `source` que dispara ação é armadilha, porque quem carrega não espera efeito colateral; (2) protege-se contra **carregamento duplo** com `[[ -n "${_LIB_X_CARREGADA:-}" ]] && return 0`, sem o que duas bibliotecas que carregam uma terceira redefinem tudo e o `readonly` explode; (3) **prefixa os nomes** (`log_info`, `http_get`), porque o Bash tem um espaço de nomes só; (4) **não tem shebang**, e sim `# shellcheck shell=bash` para o analisador saber o dialeto; (5) é carregada por **caminho absoluto calculado**, com verificação explícita de existência.

## FUNÇÕES, BIBLIOTECAS E CONFIGURAÇÃO NA PRÁTICA

kali@kali: ~/lab-estrutura

```
# a guarda que faz o mesmo arquivo ser programa E biblioteca
$ if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then principal "$@"; fi
# a ajuda sai do próprio cabeçalho — nunca dessincroniza
$ uso() { sed -n '2,9p' "${BASH_SOURCE[0]}" | sed 's/^# \?//'; }
# carregar com mensagem clara em vez de erro obscuro do set -e
$ carregar() {
    local arq="$RAIZ/lib/$1"
    [[ -r "$arq" ]] || { printf '%s: biblioteca ausente: %s\n' \
        "$PROGRAMA" "$arq" >&2; exit 70; }
    # shellcheck source=/dev/null
    source "$arq"
  }
$ head -3 lib/log.sh
# shellcheck shell=bash
[[ -n "${_LIB_LOG_CARREGADA:-}" ]] && return 0
readonly _LIB_LOG_CARREGADA=1
# devolver ESTRUTURA sem subshell e sem serializar em texto
$ carregar_config() { local -n _ref=$1; _ref[host]='exemplo.local'; }
$ declare -A cfg; carregar_config cfg; echo "${cfg[host]}"
exemplo.local
# o teste carrega a lib e executa o bin — a seta da figura, na prática
$ bats tests/
6 tests, 0 failures
```

### FUNÇÕES: AS REGRAS

| | |
|---|---|
| `local` | **toda** variável de dentro. Sem isso o `i` do laço interno destrói o `i` do chamador, e o sintoma aparece **longe da causa**. `local -r` constante, `-a` array, `-i` inteiro |
| `_prefixo` | função interna de biblioteca — sinaliza que não é interface pública |
| `status` | devolver **sim ou não**: `eh_valido "$v" \|\| die …` |
| `stdout` | devolver **um valor**, capturado com `$( )` |
| `local -n` | devolver **estrutura**: cria referência ao array do chamador, sem **fork**. Prefixe o parâmetro (`_ref`) ou colide com o nome de fora |
| `contrato` | três linhas de comentário: assinatura, o que faz, e **quais status devolve** |

### CONFIGURAÇÃO: DA MENOR À MAIOR PRIORIDADE

1. padrões no próprio script
2. `/etc/meuprog/config`
3. `${XDG_CONFIG_HOME:-$HOME/.config}/meuprog/config`
4. variáveis de ambiente (`MEUPROG_DESTINO`)
5. **opções de linha de comando** — analisadas por último, sobrepõem tudo

**Saiba quando parar de usar shell — é o ponto que teria salvado o `deploy.sh`.** O shell é excelente em orquestrar processos, encadear ferramentas e manipular arquivos, e ruim em tudo o mais. Os sinais de que o limite foi ultrapassado: **estrutura de dados aninhada** (simular objeto dentro de lista com nomes de variáveis concatenados é o sinal mais claro de todos); **aritmética não inteira**, que obriga a chamar `bc` ou `awk` a cada operação; **tratamento de erro por item** com retentativa e registro, que sai com três vezes o tamanho do equivalente em Python; **concorrência com estado compartilhado**; **passar de 300 ou 400 linhas** sem que a maior parte seja invocação de comando externo; e **precisar de testes de unidade extensos**. A migração não é total: mantenha o shell como **casca** — analisar opções, verificar dependências, orquestrar — e delegue a lógica com `exec python3 "$RAIZ/lib/processar.py" --ambiente "$AMBIENTE"`. O `exec` **substitui** o processo do shell, então sinais e códigos de saída se propagam sem uma camada no meio. **E se você herdou o arquivo de 1.400 linhas,**